

Aliro Certification Tool

User Manual

Connectivity Standards Alliance – Copyright Notice, License and Disclaimer

Copyright © Connectivity Standards Alliance (2026). All Rights Reserved. The information within this document is the property of the Connectivity Standards Alliance and its use and disclosure are restricted, except as expressly set forth herein.

Connectivity Standards Alliance hereby grants you a fully-paid, non-exclusive, non-transferable, worldwide, limited and revocable license (without the right to sublicense), under Connectivity Standards Alliance's applicable copyright rights, to view, download, save, reproduce and use the document solely for your own internal purposes and in accordance with the terms of the license set forth herein. This license does not authorize you to, and you expressly warrant that you shall not: (a) permit others (outside your organization) to use this document; (b) post or publish this document; (c) modify, adapt, translate, or otherwise change this document in any manner or create any derivative work based on this document; (d) remove or modify any notice or label on this document, including this Copyright Notice, License and Disclaimer. The Connectivity Standards Alliance does not grant you any license hereunder other than as expressly stated herein.

Elements of this document may be subject to third party intellectual property rights, including without limitation, patent, copyright or trademark rights, and any such third party may or may not be a member of the Connectivity Standards Alliance. Connectivity Standards Alliance members grant other Connectivity Standards Alliance members certain intellectual property rights as set forth in the Connectivity Standards Alliance IPR Policy. Connectivity Standards Alliance members do not grant you any rights under this license. The Connectivity Standards Alliance is not responsible for, and shall not be held responsible in any manner for, identifying or failing to identify any or all such third party intellectual property rights. Please visit www.csa-iot.org for more information on how to become a member of the Connectivity Standards Alliance.

This document and the information contained herein are provided on an “AS IS” basis and the Connectivity Standards Alliance DISCLAIMS ALL WARRANTIES EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO (A) ANY WARRANTY THAT THE USE OF THE INFORMATION HEREIN WILL NOT INFRINGE ANY RIGHTS OF THIRD PARTIES (INCLUDING WITHOUT LIMITATION ANY INTELLECTUAL PROPERTY RIGHTS INCLUDING PATENT, COPYRIGHT OR TRADEMARK RIGHTS); OR (B) ANY IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE OR NONINFRINGEMENT. IN NO EVENT WILL THE CONNECTIVITY STANDARDS ALLIANCE BE LIABLE FOR ANY LOSS OF PROFITS, LOSS OF BUSINESS, LOSS OF USE OF DATA, INTERRUPTION OF BUSINESS, OR FOR ANY OTHER DIRECT, INDIRECT, SPECIAL OR EXEMPLARY, INCIDENTAL, PUNITIVE OR CONSEQUENTIAL DAMAGES OF ANY KIND, IN CONTRACT OR IN TORT, IN CONNECTION WITH THIS DOCUMENT OR THE INFORMATION CONTAINED HEREIN, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH LOSS OR DAMAGE.

All company, brand and product names in this document may be trademarks that are the sole property of their respective owners.

This Copyright Notice, License and Disclaimer must be included on all copies of this document.

Revision History

Rev	Date	Description
1	02/09/2026	Initial revision

Table of Contents

1. Introduction	6
2. References	6
3. Aliro Test Harness Overview	7
3.1 Architecture and Layout.....	7
3.2 Data Model	8
3.3 Data Flow	8
4. Getting Started	9
4.1 System Requirements	9
4.2 Installing Ubuntu on SD-Card.....	10
4.3 Flashing Murata UWB Board.....	10
4.4 Assembling the Raspberry Pi	10
4.5 Connecting to the Raspberry Pi	11
4.6 Installing the Aliro Certification Tool	12
4.7 Initial Setup and Start-up	12
5. Using the Web GUI.....	14
5.1 Opening the GUI.....	14
5.2 Creating and Configuring a Test Project.....	14
5.3 Creating and Running a Test Run	15
6. Test Parameters.....	16
6.1 Test Parameters for Reader Tests	16
6.2 Test Parameters for User Device Tests.....	17
7. Updating and Managing the Tool	18
7.1 Updating to a New Release	18
7.2 Starting and Stopping the Test Harness	18
7.3 Accessing Logs	18
7.4 Optional Network Configurations	19
8. Authoring and Maintaining Test Scripts.....	20
9. Step-Up Provisioning	21

10. Aliro Actuator	22
10.1 Overview	22
10.2 Hardware and Prerequisites	22
10.3 NFC Support (NCI Library).....	22
10.4 Python Environment (Poetry)	23
10.5 Firmware and Flashing.....	23
10.6 Examples.....	23
10.7 Using the Actuator API	24
10.8 Packaging, Development, and Quality.....	24
11. Actuator Integration with the Test Harness.....	25
11.1 Typical Test Setups	25
11.2 Mapping Test Parameters to Actuator Configuration.....	25
11.3 Example Integration Flows	26
11.4 Using Actuator Helpers to Fill Test Parameters.....	26
11.5 Summary Mapping	26

1. Introduction

The Aliro Certification Tool is a test harness and tooling environment designed to simplify development, testing, and certification of Aliro devices, as defined by the Connectivity Standards Alliance Access Control Working Group (ACWG).

This tool provides:

- A browser-based user interface to configure test projects.
- Integration with Aliro-specific test scripts and the Aliro Actuator.
- Automated and semi-automated execution of test cases based on the Aliro specification and ACWG CSG Test Plan.

This version of the Aliro Certification Tool uses:

- Aliro Specification Version 1.0
- Aliro Test Plan Version 1.0

Intended audience

- Device vendors implementing Aliro Readers and User Devices.
- Test and certification lab engineers.
- Internal development, QA, and pre-certification teams.

2. References

1. Aliro Specification 1.0
2. Aliro Test Plan Version 1.0
3. Aliro Certification Tool Repository:
`Aliro TestHarness_CertificationTool_v1.0.zip/aliro-certification-tool`
4. Aliro Actuator Repository and Murata Firmware Instructions:
`Aliro TestHarness_CertificationTool_v1.0.zip/aliro-actuator`
(See the `third_party` folder for Murata firmware update information.)
5. Raspberry Pi Imager:
<https://www.raspberrypi.com/software/>

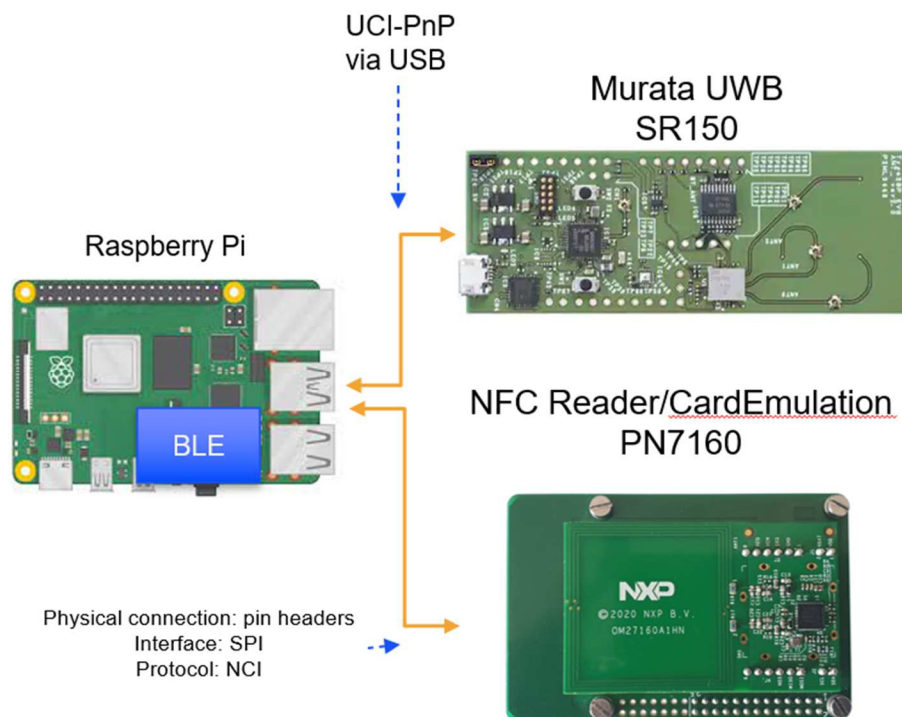
3. Aliro Test Harness Overview

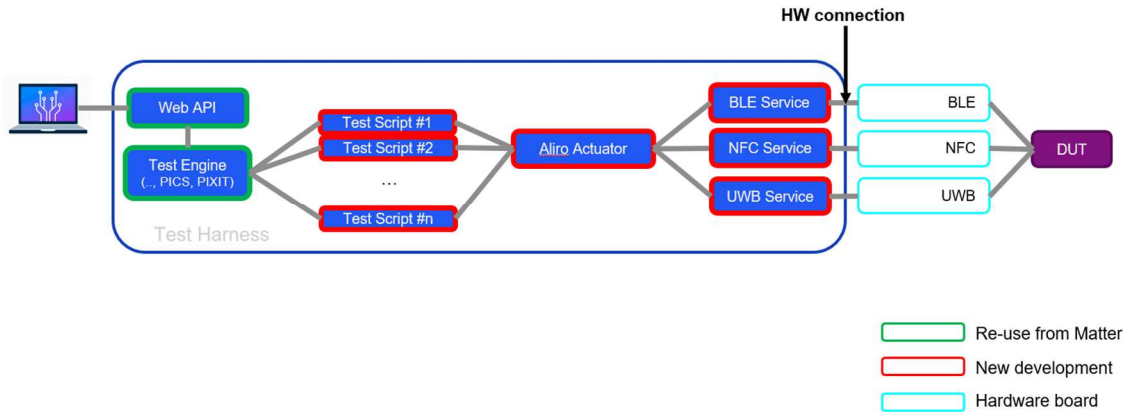
3.1 Architecture and Layout

The Aliro Certification Tool runs on a Raspberry Pi 4 with Ubuntu Server 22.04.x LTS (64-bit) and consists of:

- **Core Test Harness containers**
 - Backend services (logic, scheduling, orchestration).
 - Web frontend (GUI).
 - Database (projects, test runs, configurations).
- **Aliro test collections**
 - Located in `test_collections/aliro`.
 - The `setup.sh` script builds Aliro-specific components, including NFC drivers.
- **Hardware interfaces**
 - NFC: NXP PN7160 evaluation kits
 - OM27160A1EVK (I2C) or OM27160B1EVK (SPI).
 - BLE/UWB: Murata LBUA0VG2BP-EVK-P via micro-USB.
- **System service**
 - A systemd service (e.g. `aliro-th`) starts the Test Harness automatically at boot.

Here below is an illustration of the set-up:





3.2 Data Model

Key data entities:

- **Project**
 - Represents a device or campaign under test.
 - Stores Aliro test parameters (keys, identifiers, certificates).
- **Test Suite**
 - Group of related test cases (e.g., Reader tests, User Device tests, step-up tests).
- **Test Case**
 - Individual test script, mapped to ACWG test plan cases.
- **Test Run**
 - Execution of selected test cases under a specific project configuration.
 - Records operator, time, verdicts, and logs.
- **Test Parameters**
 - Aliro-specific cryptographic and identity data used by scripts.

3.3 Data Flow

A typical flow:

1. User creates a **Project** and configures test parameters.
2. User creates a **Test Run** with selected test suites/cases.
3. The harness orchestrates scripts that use the Aliro Actuator as protocol peer.
4. Test results and logs are stored and accessible via the GUI or CLI.

4. Getting Started

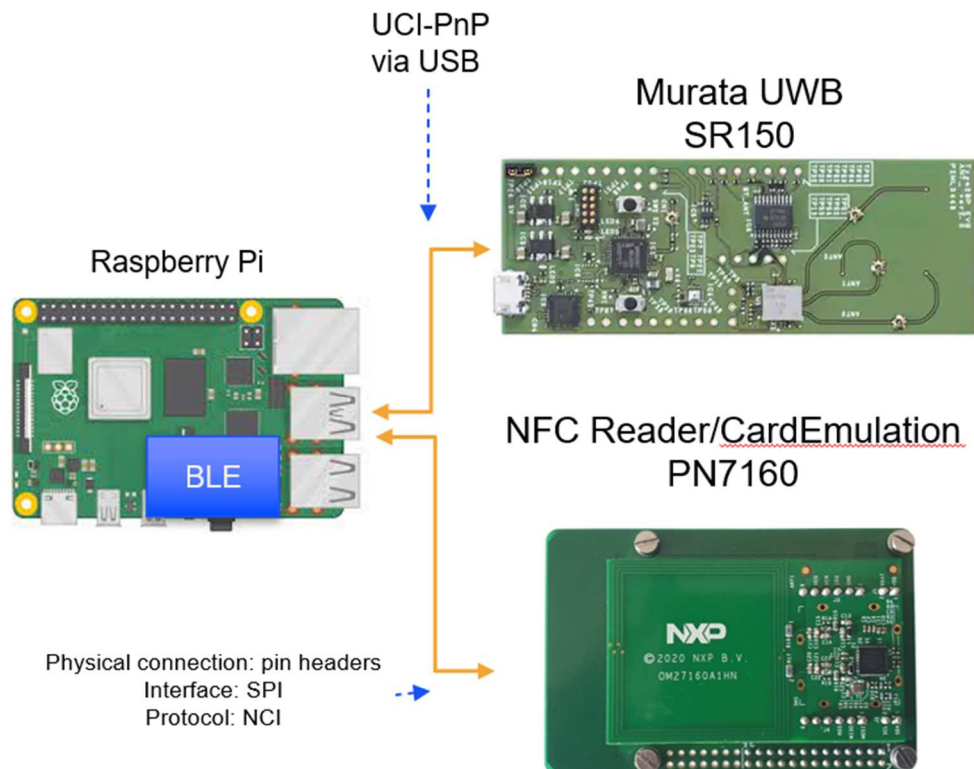
4.1 System Requirements

Hardware

- Raspberry Pi 4 Model B
 - 8 GB RAM recommended (4 GB may work).
 - Micro SD card: 16 GB or more.
 - Power supply.
- NFC interface:
 - OM27160A1EVK (I2C) or OM27160B1EVK (SPI).
- BLE/UWB interface:
 - Murata LBUA0VG2BP-EVK-P + micro-USB to USB-A cable.
- Network:
 - LAN/Wi-Fi network with Internet access.
 - Optional Ethernet cable.

Optional:

- Micro-HDMI to HDMI cable.
- Monitor and USB keyboard.



4.2 Installing Ubuntu on SD-Card

1. Download and run Raspberry Pi Imager:
<https://www.raspberrypi.com/software/>
2. CHOOSE DEVICE: select “Raspberry Pi 4”.
3. CHOOSE OS → Other general-purpose OS → Ubuntu → select one of:
 - Ubuntu Server 22.04.3 LTS (64-bit)
 - Ubuntu Server 22.04.4 LTS (64-bit)
 - Ubuntu Server 22.04.5 LTS (64-bit)

Important: You must pick exactly one of the above versions.

4. CHOOSE STORAGE: select the micro SD card.
5. Click NEXT.
6. In OS Customization:
 - Set hostname (e.g. `aliro-th-pil`).
 - Set username and password.
 - Optionally configure Wi-Fi (only password-protected networks).
 - Enable SSH with password authentication.
7. Save settings and write the OS to the SD card.
8. Wait for writing and verification to complete.

4.3 Flashing Murata UWB Board

Before use with the Test Harness, the Murata board firmware needs to be updated. The firmware file is named “`uwb_ble_device_fw-v05.00.03.bin`” (or a later version) and can be found at <https://github.com/NXP/aliro-th-additions>. The board can be updated with the DK6Programmer tool (Windows only), or with MCUxpresso and a debugger.

For more details, see https://github.com/csa-access-control/aliro-actuator/blob/main/third_party/readme.md

4.4 Assembling the Raspberry Pi

1. Ensure the Raspberry Pi is powered off.
2. Attach OM27160 evaluation kit to the Raspberry Pi as per its hardware guide.
3. Connect Murata LBUA0VG2BP-EVK-P via the micro-USB cable.
4. Insert the micro SD card.
5. Optionally connect Ethernet, monitor, and keyboard.

6. Power on the Raspberry Pi.

4.5 Connecting to the Raspberry Pi

4.5.1 SSH using Hostname

1. Wait for the Pi to boot.
2. On your PC:

```
sh
ssh <username>@<hostname>.local
# Example:
ssh ubuntu@aliro-th-pi1.local
```

3. Enter the password.

If hostname resolution fails, use the IP address.

4.5.2 SSH using IP Address

1. Wait for the Pi to boot.
2. On your PC:

```
sh
ssh <username>@<ip-address>
# Example:
ssh ubuntu@192.168.2.9
```

3. Enter the password.

To discover the IP:

- On Linux/macOS:

```
sh
arp -na | grep -i "b8:27:eb\|dc:a6:32\|e4:5f:01"
```

- On Windows:

```
bat
arp -a | findstr b8-27-eb dc-a6-32 e4-5f-01
```

4.5.3 Using Monitor and Keyboard

Log in locally with the configured username and password (default Ubuntu: ubuntu/ubuntu if unchanged).

4.6 Installing the Aliro Certification Tool

Run the auto-installer:

```
sh
cd ~/aliro-certification-tool
./scripts/pi-setup/auto-install.sh
```

Enter your password when prompted. At the end, choose option 1 to reboot.

Notes:

- Installation can take over an hour, depending on network speed.
- First reboot after installation may take 5+ minutes.

4.7 Initial Setup and Start-up

1. Initialize submodules:

```
sh
cd ~/aliro-certification-tool
git submodule update --init --recursive
```

2. Set up Aliro test collection:

- SPI kit:

```
sh
cd ~/aliro-certification-tool/test_collections/aliro
./setup.sh
```

- I2C kit:

```
sh
cd ~/aliro-certification-tool/test_collections/aliro
NXP_TRANSPORT=I2C ./setup.sh
```

3. Start the harness:

```
sh
cd ~/aliro-certification-tool
./scripts/start.sh
```

The harness also starts automatically at boot.

5. Using the Web GUI

5.1 Opening the GUI

1. Ensure your PC and the Raspberry Pi are on the same network.
2. In a browser, navigate to:

`http://<raspberry-pi-ip-address>`

Example: `http://192.168.2.9`
3. Wait a couple minutes after boot before connecting.

To find the Pi IP on the Pi:

```
sh
hostname -I
```

5.2 Creating and Configuring a Test Project

1. Click “Create Project”.
2. Enter a project name.
3. Click “Edit” to open the JSON configuration.
4. Locate the "test_parameters" field (default null).
5. For Reader tests, set parameters such as:

```
json
{
  "test_parameters": {
    "dut_reader_public_key": "043928f3...c5d34ee",
    "dut_reader_group_identifier":
"00113344667799AA00113344667799AA",
    "dut_reader_issuer_group_identifier":
"00113344667799AA00113344667799AB",
    "dut_reader_group_sub_identifier":
"113344667799AA00113344667799AA00",
    "dut_reader_group_resolving_key":
"00000000000000000000000000000000"
  }
}
```

6. Click “Update” to save.
7. Click “Create” to create the project.

You can edit parameters later by clicking the pencil icon on the project row.

5.3 Creating and Running a Test Run

1. On the projects list, click the triangular “Go To Test-Run” icon next to the project.
2. Click “Create new Test Run”.
3. Choose or create an “Operator Name” in the top right.
4. Select one or more test suites or individual test cases.
5. Click “Start” to execute.

The GUI displays test status and logs in real time.

6. Test Parameters

The Aliro test scripts depend on cryptographic keys, group identifiers, and certificates configured in `test_parameters`.

You can configure them at project creation or later by editing the project.

6.1 Test Parameters for Reader Tests

- `dut_reader_public_key`
Public key of the Reader DUT.
Format: DER-encoded HEX or PEM with `\n`.
- `dut_reader_group_identifier`
Group Identifier of the Reader DUT.
Format: HEX string.
- `dut_reader_issuer_group_identifier`
Group Identifier for Reader Issuer CA certificate.
Format: HEX string.
- `dut_reader_group_sub_identifier`
Sub-group Identifier for Reader DUT.
Format: HEX string.
- `dut_reader_group_resolving_key`
Group resolving key used in BLE tests.
Format: HEX string.
- `th_access_credential_private_key / th_access_credential_public_key`
Keys for the user access credential simulated by the tool.
Format: DER HEX or PEM with `\n`.
- `dut_reader_issuer_public_key`
Reader System Issuer CA certificate public key (for certificate verification).
Format: DER HEX or PEM with `\n`.
- `th_credential_issuer_private_key / th_credential_issuer_public_key`
Keys used to sign simulated Access Documents for the Reader.
Format: DER HEX (for private key: 138 or 32 bytes) or PEM with `\n`.
- `th_credential_issuer_ca_private_key / th_credential_issuer_ca_public_key`
Keys used to sign simulated Access Document Certificates for the Reader.
Format: DER HEX (for private key: 138 or 32 bytes) or PEM with `\n`.
- `dut_access_element_id`
Access Element ID requested by the Reader.
Format: string.

6.2 Test Parameters for User Device Tests

Private and public key fields must match: either configure both or none.

- `th_reader_private_key/th_reader_public_key`
Keys for the simulated Reader.
Format: DER HEX (for private key: 138 or 32 bytes) or PEM with \n.
- `th_reader_group_identifier`
Group Identifier for simulated Reader.
Format: HEX string.
- `th_reader_sub_group_identifier`
Sub-group Identifier for simulated Reader.
Format: HEX string.
- `th_reader_certificate`
Reader certificate used for LOAD CERT and AUTH1 commands.
Format: HEX string.
- `th_reader_group_resolving_key`
Group resolving key used for BLE tests.
Format: HEX string.
- `th_reader_spsm`
SPSM value for BLE tests.
Format: HEX string.
- `th_access_credential_public_key`
Access credential public key for the key slot lookup.
Format: DER HEX or PEM with \n.
- `th_reader_issuer_public_key`
Reader System Issuer CA certificate public key for key generation.
Format: DER HEX or PEM with \n.
- `dut_credential_issuer_public_key`
Credential Issuer Public key loaded into the Reader DUT to validate Access Documents.
Format: DER-encoded HEX or PEM with \n.
- `dut_credential_issuer_ca_public_key`
Credential Issuer CA Public key loaded into the Reader DUT to validate Access Document Certificates.
Format: DER-encoded HEX or PEM with \n.
- `th_access_element_id`
Access Element ID requested by simulated Reader.
Format: string.

7. Updating and Managing the Tool

7.1 Updating to a New Release

1. On the Pi:

sh
<pre>cd ~/aliro-certification-tool git fetch git checkout <target-release-tag> # Example: git checkout release/test_event1-2024</pre>

2. Run the update script:

sh
<pre>./scripts/update.sh</pre>

7.2 Starting and Stopping the Test Harness

- Stop:

sh
<pre>cd ~/aliro-certification-tool ./scripts/stop.sh</pre>

- Start:

sh
<pre>./scripts/start.sh</pre>

To disable autostart:

sh
<pre>sudo systemctl disable aliro-th</pre>

7.3 Accessing Logs

From the `aliro-certification-tool` directory:

sh
<pre>docker compose logs</pre>

Use `docker compose logs -f` to follow logs live.

7.4 Optional Network Configurations

7.4.1 Wi-Fi Without Password

Edit `/etc/netplan/50-cloud-init.yaml`:

yaml

```
network:
  ...
  wifis:
    wlan0:
      dhcp4: true
      optional: true
      access-points:
        "<network_name>": {}
```

Apply and reboot:

sh

```
sudo netplan apply
sudo reboot
```

7.4.2 Link-Local Address on Ethernet

Edit `/etc/netplan/50-cloud-init.yaml`:

yaml

```
network:
  ...
  ethernet:
    eth0:
      dhcp4: true
      optional: true
      link-local: [ ipv4, ipv6 ]
```

Apply and reboot:

sh

```
sudo netplan apply
sudo reboot
```

8. Authoring and Maintaining Test Scripts

Aliro test scripts live under:

text

`test_collections/aliro`

They must follow the same structure as the sample collections (suites and test cases) so the harness can auto-discover them.

After modifying or adding test scripts:

1. Restart the backend container:

sh
<code>docker restart aliro-certification-tool_backend_1</code>

2. View backend logs with:

sh
<code>docker compose logs backend</code>

(adapt service name if different.)

9. Step-Up Provisioning

Step-up tests use three components:

- Access Document
- Device Request
- Device Response

Based on the Device Request, a Device Response is constructed using the Access Document. To validate step-up tests, the DUT must contain the correct Access Document.

A provisioning script is provided at:

text
<code>test_collections/aliro/support/access_doc/step-up/step_up_provision.py</code>

Use this script to prepare and load the correct Access Document in the DUT before running step-up certification tests.

10. Aliro Actuator

10.1 Overview

The Aliro Actuator is a Python implementation of the Aliro specification used as:

- Reference implementation of Reader and User Device roles.
- Programmable peer for certification tests (controlled via scripts).
- Tool for generating keys and certificates.

Project documentation (HTML/PDF) is under `docs/build`. Examples are under `examples`.

10.2 Hardware and Prerequisites

The actuator uses the NXP `nci` library for PN7160 NFC boards:

- OM27160A1EVK (I2C)
- OM27160B1EVK (SPI)

For BLE/UWB, the Murata LBUA0VG2BP-EVK-P must have the correct firmware (see `third_party/murata_fw`).

10.3 NFC Support (NCI Library)

Default installation:

- SPI kit:

sh
<code>./scripts/install_nfc.sh</code>

- I2C kit:

sh
<code>NXP_TRANSPORT=I2C ./scripts/install_nfc.sh</code>

If needed, manual build via `linux_libnfc-nci` (clone, edit `libnfc-nxp.conf`, apply patch, configure, make, install).

10.4 Python Environment (Poetry)

Install dependencies:

sh
<pre>sudo apt install python3-pip pip install poetry PATH="\$PATH:/home/<user-name>/.local/bin" sudo -E \$(which poetry) install --no-root sudo -E \$(which poetry) shell</pre>

10.5 Firmware and Flashing

Flash recommended firmware for Murata and other boards as described in `third_party/readme.md`.

10.6 Examples

Inside the Poetry shell:

- NFC:

sh
<pre>python3 -m examples.nfc.reader.standard python3 -m examples.nfc.reader.fast python3 -m examples.nfc.user_device</pre>

- BLE:

sh
<pre>python3 -m examples.ble.reader.standard python3 -m examples.ble.reader.fast python3 -m examples.ble.user_device</pre>

- UWB:

sh
<pre>python3 -m examples.ble.reader.ble_uwb python3 -m examples.ble.user_device_ble_uwb</pre>

- Cryptography:

sh
<pre>python3 -m examples.cryptography.generate_certificate python3 -m examples.cryptography.generate_keypair</pre>

10.7 Using the Actuator API

- Reader class: `aliro_actuator.access_protocol.reader.Reader`
- User Device class: `aliro_actuator.access_protocol.user_device.UserDevice`

Common usage:

python

```
reader = Reader(transport_protocol="NFC") # or BLE/UWB
reader.transaction_initiation()
```

or:

python

```
user = UserDevice(transport_protocol="BLE")
user.transaction_initiation()
```

Two patterns:

1. **Handle functions (session-based)**
 - Call `start_new_session()`.
 - Use methods prefixed with `handle_...`
 - State is stored in a session object.
2. **Command/response methods (low-level)**
 - Use `command_...` and `response_...` methods.
 - Provide all fields explicitly and handle responses manually.

10.8 Packaging, Development, and Quality

- Build wheel:

sh

```
poetry build
```

- VS Code + Dev Container:
 - Open repo, reopen in Dev Container, press F5 to debug.

- Testing:

sh
scripts/test.sh

- Linting and formatting:

sh
scripts/lint.sh
scripts/format.sh

Quality tools: flake8, black, isort, mypy, cspell.

11. Actuator Integration with the Test Harness

11.1 Typical Test Setups

1. Reader DUT

- DUT implements Reader.
- Actuator simulates User Device.
- Test Harness coordinates scripts and uses “Reader test parameters”.

2. User Device DUT

- DUT implements User Device.
- Actuator simulates Reader.
- Test Harness coordinates scripts and uses “User Device test parameters”.

11.2 Mapping Test Parameters to Actuator Configuration

Reader DUT

Use parameters described in Section 6.1:

- `dut_reader_*` describe the Reader DUT.
- `th_access_credential_*` describe the simulated User Device.
- `dut_reader_issuer_public_key` validates Reader certificates.

User Device DUT

Use parameters described in Section 6.2:

- `th_reader_*` configure the simulated Reader (actuator).
- `th_access_credential_public_key`, `th_reader_issuer_public_key` configure access credentials and issuer CA.

11.3 Example Integration Flows

Reader DUT, NFC Standard Access

- Hardware: Reader DUT + PN7160 with Pi.
- Actuator: `examples.nfc.user_device` (User Device role).
- Test Harness: fill `dut_reader_*` and `th_access_*` parameters, run Reader test suite.

User Device DUT, BLE Fast Transaction

- Hardware: User Device DUT + Murata board on Pi.
- Actuator: `examples.ble.reader.fast` (Reader role).
- Test Harness: fill `th_reader_*`, `th_access_*` parameters, run User Device test suite.

11.4 Using Actuator Helpers to Fill Test Parameters

Generate keys and certificates:

sh
<pre>python3 -m examples.cryptography.generate_keypair python3 -m examples.cryptography.generate_certificate</pre>

Paste generated values into `test_parameters` fields.

11.5 Summary Mapping

DUT Role	Actuator Role	Example Scripts	Main Test Parameters
Reader DUT	User Device	NFC/ BLE user_device examples	<code>`dut_reader_*</code> , <code>`th_access_*</code>
User Device	Reader	NFC/ BLE reader.* examples	<code>`th_reader_*</code> , <code>`th_access_*</code>
Step-up DUT	Reader / UD	<code>`reader/stepup.py`</code>	Above + Access Document via step-up script